DOSSIER D’ARCHITECTURE • V1.0

Organisation Development Standard

Extension VS Code pour la standardisation du développement, l’IA gouvernée, la qualité, la sécurité applicative et GitFlow.

PÉRIMÈTRE 5 onglets métier · Cœur Java 21 · Politique GitLab signée · Intégrations Mistral, SonarQube et JFrog Xray

Date : 28 octobre 2026 | Nature : conception et architecture cible | Statut : proposition technique
Nom de projet : organisation-developpent-standard

Résumé exécutif

Organisation Development Standard (ODS) est une extension VS Code destinée à centraliser les pratiques de développement approuvées par l'organisation. Elle propose cinq onglets : Project Initializer, IA Code Generator, Quality, Application Security et GitFlow. L'ensemble s'appuie sur un moteur métier Java et une gateway organisationnelle sécurisée.

Le point architectural déterminant est la ségrégation IA par projet. La politique d'autorisation est publiée par les administrateurs sur GitLab, signée, puis interprétée côté Java. Les fonctions IA sont désactivées pour les projets non éligibles ; les contrôles d'accès sont également imposés par la gateway avant chaque requête, sans dépendre uniquement de l'état visuel du plugin.

PRINCIPE DIRECTEUR Java porte la ségrégation, l'intégrité, l'orchestration et les traitements ; TypeScript est limité à l'intégration native et aux vues de VS Code. Les secrets techniques ne sont jamais livrés dans le VSIX.

Sommaire

1. Vision et expérience développeur
2. Architecture globale et responsabilités
3. Les cinq onglets fonctionnels
4. Ségrégation IA par projet GitLab
5. Intégrité des configurations et gestion des identités
6. Implémentation Java : socle de ségrégation
7. Gouvernance des prompts, agents et tokens
8. Structure du projet et contrats API
9. Technologies et exigences minimales
10. Lots de réalisation et stratégie de tests
11. Décisions d'architecture
12. Sources et documentation officielle

1. Vision et expérience développeur

L'extension vise à proposer une plateforme intégrée pour créer des projets selon les quatre lignes standard de l'organisation (Java, Python, .NET, Angular), consulter une IA approuvée, corriger les anomalies de qualité et de sécurité et exécuter les opérations GitFlow.

Les paramètres organisationnels, instructions système, URLs d'outils et autorisations sont administrés centralement. Le développeur utilise les capacités approuvées, sans pouvoir modifier l'endpoint IA ni injecter de comptes de service personnels dans les fonctions administrées.

Présentation des cinq vues

Vue	Fonction principale	État si IA interdite
Project Initializer	Démarrage depuis des starter kits officiels	Disponible
IA Code Generator	Chat, génération, refactoring et agents approuvés	Chat et exécution grisés
Quality	Scan SonarQube et recommandations	Scan disponible, remédiation IA bloquée
Application Security	Scan Xray et analyse de dépendances	Scan disponible, remédiation IA bloquée
GitFlow	Branches, commits, push et merge requests	Git disponible, rédaction IA bloquée

Dans le bandeau commun, afficher : projet GitLab détecté, branche active, stack, statut d'autorisation IA, modèle autorisé et quota restant. L'onglet IA présente un champ de chat, les éléments du contexte transmis et un indicateur de consommation des tokens.

Maquette de l'interface proposée

Capture de référence : cinq onglets, contexte du projet, autorisation IA et suivi des tokens (données fictives).

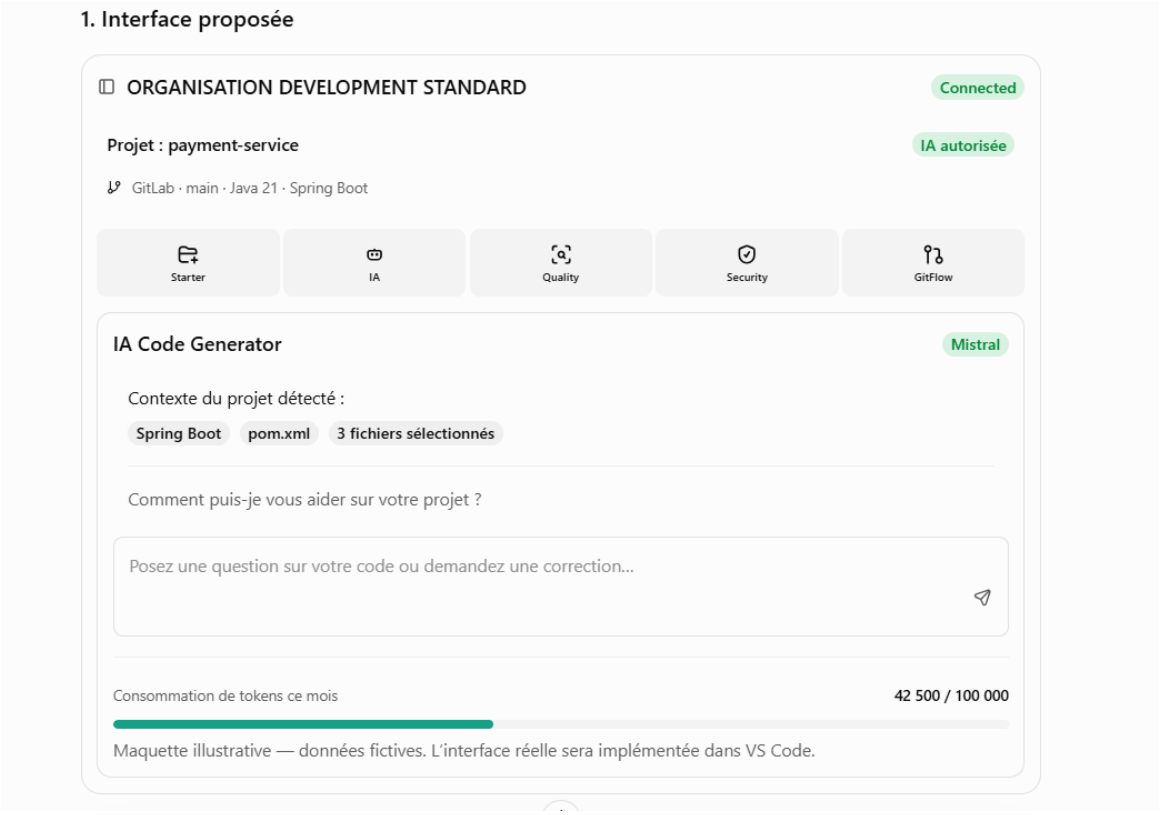
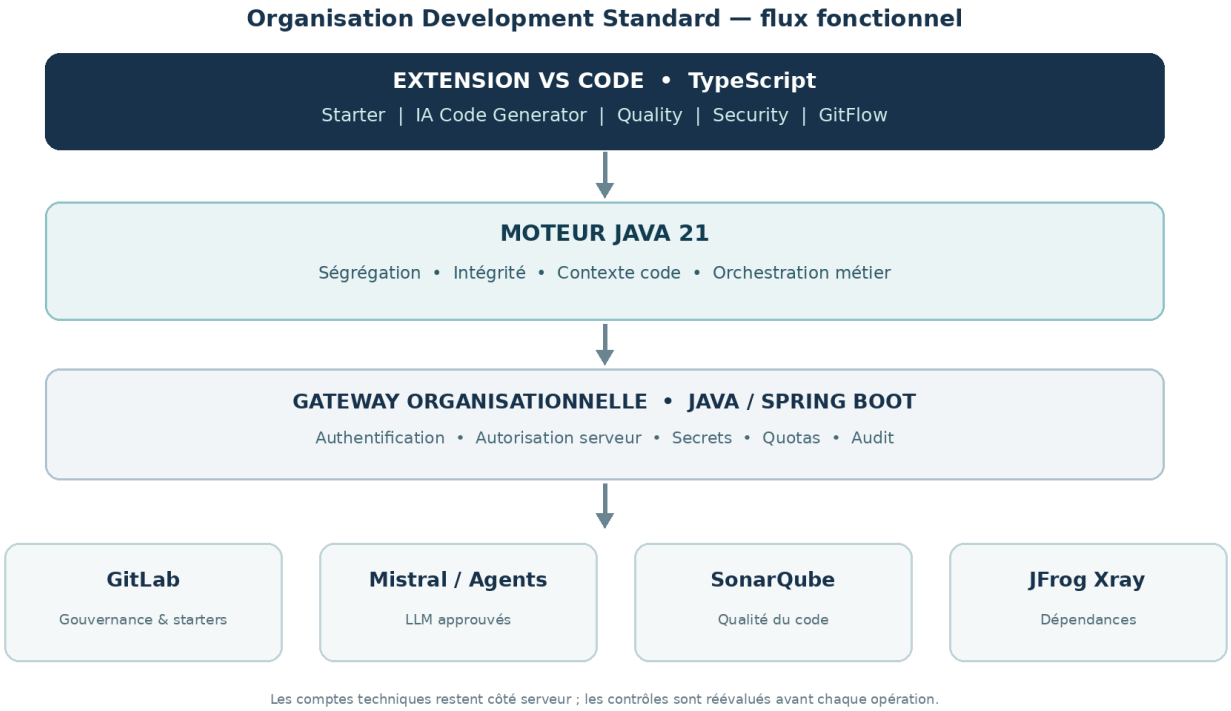


Figure — Maquette de l'interface proposée (illustration fournie ; données d'exemple).

2. Architecture globale et responsabilités



L’interface de l’extension échange avec un service Java local, via JSON-RPC sur stdio ou un loopback HTTP protégé. Le service local communique en HTTPS avec la gateway Java/Spring Boot de l’organisation. Les services distants incluent GitLab, Mistral ou un autre modèle approuvé, SonarQube, JFrog Xray et un coffre de secrets.

Composant	Technologie	Responsabilité
Extension	TypeScript + API VS Code	Vues, éditeur, sélection, diff, affichage des résultats
Moteur local	Java 21	Détection du projet, contexte, orchestration, règles locales
Gateway	Java 21 / Spring Boot	Autorisation effective, outils, identités, quotas, audit
Repository gouvernance	GitLab	Politiques IA, manifests de standards, prompts
Secrets	Vault / coffre interne	Stockage et rotation des comptes techniques
Données centrales	PostgreSQL	Historique, quotas, consommation, traçabilité

CONTRÔLE CRITIQUE Un utilisateur peut modifier une extension installée localement. La gateway doit revérifier l’identité, les droits du projet et les opérations demandées avant chaque appel sensible.

3. Les cinq onglets fonctionnels

3.1. Project Initializer

Initialise un projet à partir d’un starter kit validé et versionné dans GitLab. Le développeur choisit la ligne de développement et, le cas échéant, la variante et la version autorisée. Les conventions internes sont appliquées automatiquement.

Ligne	Éléments couverts
Java	Spring Boot, JDK, Maven/Gradle, tests, sécurité, conventions et pipeline
Python	Packages, environnement virtuel, dépendances, linting, tests et pipeline
.NET	Solution C#, ASP.NET Core, NuGet interne, tests et analyseurs
Angular	Structure du projet, npm interne, linting, tests et pipeline

Livrables générés : arborescence standard, .gitignore, README, configuration des registres internes de dépendances, conventions de nommage, CI/CD, fichiers de qualité et paramètres de sécurité. Les versions exactes sont gouvernées par manifeste organisationnel, non figées dans l'extension.

3.2. IA Code Generator

Une interface de type Continue/Codex permet au développeur de dialoguer avec un LLM déjà approuvé, tel que Mistral, et de déclencher des agents ou prompts métier préconfigurés. Le développeur ne peut ni choisir une API non approuvée, ni modifier l'authentification ou les instructions système.

- Questions contextualisées sur le fichier actif, la sélection et les fichiers pertinents du workspace.
- Génération, explication, revue, refactoring et proposition de correctifs sous forme de diff.
- Exécution d'agents autorisés avec validation des actions susceptibles de modifier le code ou d'exécuter des commandes.
- Affichage des tokens d'entrée et de sortie, du total par session, du quota restant et de la consommation par projet.
- Respect des exclusions de contexte, de la confidentialité et des restrictions de transfert de sources.

Le moteur Java prépare un contexte pertinent, limite sa taille, sélectionne les données dont l'envoi est autorisé et transmet la requête à la gateway. Le modèle IA utilisé est référencé par un alias organisationnel : un changement de modèle ne nécessite pas nécessairement une nouvelle publication du VSIX.

3.3. Quality / SonarQube

Le module Quality doit lancer le scanner compatible avec le langage et le type de projet, puis lire les résultats SonarQube par API : anomalies, sévérités, fichiers, règles et Quality Gate. L'API de consultation des résultats ne remplace pas le scanner.

- Détection de la technologie et sélection du scanner : Maven, .NET ou SonarScanner adapté.
- Déclenchement d'une analyse, suivi du statut de tâche et récupération du rapport.
- Affichage des issues classées par gravité, règle, fichier et ligne.
- Si autorisé : génération d'un prompt de remédiation et d'un patch proposé par le LLM à partir du rapport et du code ciblé.
- Vérification et tests après correction, puis nouvelle analyse SonarQube.

Si l'IA est désactivée, la consultation du rapport et les analyses non-IA restent possibles selon les droits du projet.

3.4. Application Security / JFrog Xray

Le module analyse les dépendances embarquées dans les projets Java, Python, .NET et Angular avec les mécanismes pris en charge par JFrog Xray. Selon la situation, il peut consommer un rapport existant d'un build publié ou déclencher un audit à la demande.

- Identifier les dépendances directes et transitives et leurs versions.
- Afficher vulnérabilités, CVE, criticité, chemin de dépendance et versions corrigées disponibles.
- Transmettre au LLM approuvé, uniquement si l'IA est permise, les résultats pertinents et les fichiers manifestes du projet.
- Proposer une correction de pom.xml, requirements.txt, .csproj ou package.json sans mise à jour aveugle.
- Vérifier les versions autorisées dans les registres internes, exécuter les tests et relancer le scan.

3.5. GitFlow

GitFlow rassemble les commandes Git habituelles sous forme d'actions guidées : clone, fetch, pull, création et bascule de branche, add, commit, push et création de merge request (MR). Les conventions de branches et les politiques de protection sont celles de GitLab.

Si l'IA est autorisée, la description de MR peut être préparée à partir du diff, des commits, des résultats de tests et des éventuels rapports Sonar/Xray. La proposition inclut titre, résumé, impacts, tests, tickets associés et points d'attention pour la revue.

Le push et la MR utilisent l'identité GitLab et les droits du développeur ; un compte de service privilégié ne doit pas être utilisé pour effectuer ces opérations à sa place.

4. Ségrégation IA par projet GitLab

L'autorisation IA est déterminée prioritairement par le project_id numérique GitLab combiné à l'instance GitLab attendue. Ce critère est plus stable qu'un nom de dossier ou une URL modifiable. La politique est administrée dans un repository privé protégé par les responsables IA.

Matrice de permissions

Action	Projet autorisé	Projet non autorisé
Starter Kit	Oui	Oui
Chat IA / Code Generation	Oui	Bloqué
Scan Sonar	Oui	Oui, selon droits
Remédiation Sonar par IA	Oui	Bloquée
Scan Xray	Oui	Oui, selon droits
Remédiation Xray par IA	Oui	Bloquée
Commandes Git	Oui	Oui, selon droits
Description IA de MR	Oui	Bloquée

Exemple de politique : ai-project-policy.json

```
{
  "schemaVersion": 1,
  "policyVersion": 17,
  "issuedAt": "2026-10-28T00:00:00Z",
  "expiresAt": "2026-10-29T00:00:00Z",
  "gitlabInstance": "https://gitlab.organisation.example",
  "defaultEffect": "DENY",
  "projects": [
    {
      "projectId": 1042,
      "aiEnabled": true,
      "allowedActions": ["CHAT", "CODE_GENERATION",
        "SONAR_REMEDIATION", "XRAY_REMEDIATION",
        "MR_DESCRIPTION"],
      "modelAlias": "coding-approved",
      "monthlyTokenQuota": 1000000
    },
    {"projectId": 2075, "aiEnabled": false,
      "allowedActions": []}
  ]
}
```

Le projet 1042 dispose des permissions IA listées, tandis que le projet 2075 n'a aucun accès aux opérations IA. La valeur defaultEffect = DENY applique le refus par défaut.

Cycle de vérification

1. Ouverture de VS Code / du workspace : identification du remote et du projet.
2. Vérification du projet cible auprès de GitLab, en tenant compte de l'instance attendue.
3. Récupération côté gateway de la politique officielle à l'aide du compte technique de lecture.
4. Contrôle de la signature, de la version, de la validité temporelle et des droits.
5. Publication à l'interface des fonctionnalités accordées ; les contrôles sensibles restent réappliqués sur le serveur.
6. Nouvelle évaluation lors d'un changement de projet, d'une expiration, d'une révocation et avant toute requête IA.

La Repository Files API de GitLab permet de télécharger un fichier depuis une branche/révision précise sans cloner l'intégralité du dépôt. Cette méthode est généralement préférable au checkout complet pour lire une politique.

FAIL CLOSED Projet inconnu, politique invalide ou expirée, signature incorrecte, indisponibilité sans cache encore valable : aucun appel IA n'est autorisé. Dans un workspace multi-racines, les permissions sont évaluées séparément pour chaque projet.

Attention : un remote Git peut être modifié et un dossier local peut contenir du code provenant d'un autre projet. La provenance du contexte doit être contrôlée ; l'identifiant GitLab ne suffit pas seul à prouver l'origine des fichiers envoyés au LLM.

5. Intégrité des configurations et gestion des identités

5.1. Hash, signature et chiffrement

Mécanisme	Objectif	Limite
SHA-256	Constater les différences de contenu	Ne prouve pas seul qui a produit le fichier
AES-GCM	Confidentialité et intégrité sous secret partagé	Un secret présent côté poste peut être extrait
Ed25519	Authenticité et intégrité du manifeste	La clé publique doit être fiable et protégée contre la substitution

La livraison embarque les fichiers statiques nécessaires et un manifeste signé référençant leurs empreintes SHA-256. Le moteur Java vérifie le manifeste au démarrage et avant l'usage d'une configuration sensible. Les politiques et prompts dynamiques sont fournis depuis les repositories d'administration sous forme de données signées et versionnées.

Un attribut « lecture seule » réduit les modifications accidentelles mais ne protège pas contre l'altération d'une installation locale. Le contrôle réellement contraignant est du côté de la gateway. Pour se protéger du rejeu d'une ancienne politique encore signée, la gateway doit tenir compte des versions et dates de validité.

5.2. Répartition des comptes

Opération	Identité	Emplacement du secret
Lire la politique GitLab	Compte de service à droits minimaux	Gateway / Vault
Lancer / consulter Sonar	Compte de service Sonar	Gateway ou infrastructure CI
Lancer / consulter Xray	Compte de service JFrog	Gateway ou infrastructure CI

Opération	Identité	Emplacement du secret
Appeler Mistral / LLM	Développeur authentifié, accès délégué	SSO / Gateway, clé fournisseur côté serveur
Push / création de MR	Compte GitLab du développeur	OAuth / autorisation utilisateur

Il ne faut pas embarquer de compte technique privilégié ou de clé fournisseur dans le VSIX, même chiffrés, car l'extension peut être inspectée et son processus local compromis. Le coffre de secrets central doit gérer la rotation et la limitation des droits.

L'authentification utilisateur (OIDC/SSO) permet à la gateway de distinguer les développeurs, de gérer les quotas et de journaliser la consommation IA tout en conservant une éventuelle clé d'organisation côté serveur.

6. Implémentation Java : socle de ségrégation

Le code suivant reproduit le noyau proposé : vérification d'une signature Ed25519 sur les octets d'une politique et décision d'autorisation par project_id, action, confiance du workspace et date d'expiration. Il s'agit d'une base compilable, non d'une implémentation complète de production.

```
import java.security.GeneralSecurityException;
import java.security.KeyFactory;
import java.security.PublicKey;
import java.security.Signature;
import java.security.spec.X509EncodedKeySpec;
import java.time.Instant;
import java.util.Map;
import java.util.Set;

public final class PolicyGate {
    private PolicyGate() {}

    public enum Action {
        CHAT, CODE_GENERATION, SONAR_REMEDIATION,
        XRAY_REMEDIATION, MR_DESCRIPTION
    }

    public record Rule(
        long projectId,
        Set<Action> allowed,
        Instant expiresAt
    ) {}

    public static boolean verifyEd25519(
            byte[] policy,
            byte[] signature,
            byte[] publicKeyDer) {
        try {
            PublicKey key = KeyFactory.getInstance("Ed25519")
                .generatePublic(
                    new X509EncodedKeySpec(publicKeyDer)
                );

            Signature verifier = Signature.getInstance("Ed25519");
            verifier.initVerify(key);
            verifier.update(policy);
            return verifier.verify(signature);
        } catch (GeneralSecurityException exception) {
            return false;
        }
    }

    public static boolean allowed(
            long projectId,
            Action action,
            boolean workspaceTrusted,
```

```
            boolean signatureVerified,
            Instant now,
            Map<Long, Rule> rules) {

        if (!workspaceTrusted || !signatureVerified
                || action == null || now == null) {
            return false;
        }

        Rule rule = rules.get(projectId);

        return rule != null
            && rule.projectId() == projectId
            && rule.expiresAt() != null
            && now.isBefore(rule.expiresAt())
            && rule.allowed() != null
            && rule.allowed().contains(action);
    }
}
```

Pré-requis pour cette classe : JDK 17 au minimum pour les records et Ed25519 (JDK 21 recommandé). Compilation : javac PolicyGate.java. Aucune dépendance externe n'est nécessaire pour cet exemple. La compilation a été vérifiée durant la génération du document.

Pour la production : ajouter le parsing JSON, le schéma des permissions, la validation du domaine GitLab, le suivi de version/anti-rejeu, la gestion d'erreur, le cache avec TTL, l'audit, les tests automatisés et le contrôle équivalent sur la gateway.

7. Gouvernance des prompts, agents et tokens

7.1. Repository des prompts

Un repository protégé, par exemple organisation-development-prompts, stocke les prompts métier validés, leurs versions, les instructions système, les contraintes de sécurité et les manifestes signés. Le développeur peut saisir une question libre ou lancer un prompt approuvé, sans pouvoir substituer les instructions organisationnelles.

Prompt	Fonction
code-generation	Produire du code conforme aux conventions
code-review	Relecture ciblée de code
sonar-remediation	Aide à la correction d'issues Sonar
xray-remediation	Aide à la correction de vulnérabilités
gitlab-mr-description	Description et résumé d'une merge request

7.2. Construction du contexte

Le contexte envoyé au LLM combine les instructions système signées, une sélection de code pertinente et autorisée, et la demande utilisateur. Les fichiers locaux, rapports Sonar et résultats Xray restent des données non fiables : ils ne peuvent pas modifier les règles système. Les fichiers secrets, credentials et éléments exclus doivent être filtrés avant tout envoi.

Pour les agents, définir des permissions distinctes : lecture de fichiers, modification de code, commande shell, accès réseau. Les opérations à effet de bord (écriture, installation, suppression, push) nécessitent une politique explicite, une journalisation et, pour les actions sensibles, une confirmation du développeur.

7.3. Mesure des tokens

Afficher la consommation par requête, session, développeur et projet : tokens d'entrée, tokens de sortie, cumul mensuel, plafond autorisé et éventuel coût estimé. Le registre central doit utiliser en priorité les métriques de consommation retournées par le fournisseur IA. Le coût dépend du modèle et de la tarification en vigueur, et ne doit pas être confondu avec le nombre de tokens.

8. Structure du projet et contrats API

Arborescence cible du repository

```
organisation-developpent-standard/
├── vscode-extension/
│   ├── package.json
│   └── src/
│       ├── extension.ts
│       ├── views/
│       │   ├── StarterView.ts
│       │   ├── AiChatView.ts
│       │   ├── QualityView.ts
│       │   ├── SecurityView.ts
│       │   └── GitFlowView.ts
│       └── java-client/JavaProcessClient.ts
├── ods-java-core/
│   ├── ods-security/      # signatures, politiques
│   ├── ods-starter/       # modèles de projet
│   ├── ods-ai-context/    # préparation de contexte
│   ├── ods-quality/       # Sonar
│   ├── ods-xray/          # JFrog
│   ├── ods-gitflow/       # Git / MR
│   └── ods-config/
├── ods-gateway/
│   └── src/main/java/org/organisation/ods/
│       ├── auth/    ├── policy/  ├── gitlab/
│       ├── mistral/ ├── sonar/   ├── jfrog/
│       └── audit/   └── quota/
├── policies/             # schémas et exemples
├── starter-manifests/
├── prompt-manifests/
└── docs/
```

Le répertoire policies/ du repository d'application contient des exemples et des schémas. Les véritables politiques d'autorisation de production restent dans un repository GitLab administré distinct.

Endpoints proposés de la gateway (à développer)

Endpoint	Rôle
GET /api/v1/capabilities	Permissions utilisateur/projet
GET /api/v1/starters	Liste des templates autorisés
POST /api/v1/ai/chat	Chat contextualisé
POST /api/v1/ai/agents/run	Déclenchement d'agent approuvé
GET /api/v1/ai/usage	Tokens et quotas
POST /api/v1/quality/scans	Analyse SonarQube
POST /api/v1/security/scans	Analyse JFrog Xray
POST /api/v1/remediations	Aide à la correction des anomalies
POST /api/v1/git/merge-requests	Création de MR via identité utilisateur

Ces endpoints sont des contrats internes proposés, pas des routes natives des outils tiers. Les analyses Sonar/Xray doivent être traitées en tâches asynchrones avec identifiant, statut, récupération des résultats et annulation éventuelle. L'extension ne doit pas attendre qu'un scanner long termine dans un appel HTTP bloquant.

9. Technologies et exigences minimales

Élément	Recommandation
Extension	VS Code Desktop, API VS Code, TypeScript
JDK moteur et gateway	Java 21 recommandé
Framework backend	Spring Boot 4.x avec version compatible JDK
Build Java	Maven 3.9.x recommandé
Runtime interface	Node.js compatible avec outils d'extension / VS Code
Communication locale	JSON-RPC stdio ou loopback protégé
Communication distante	HTTPS REST, TLS organisationnel
Gestion d'identité	SSO/OIDC, RBAC et autorisation par projet
Signatures	Ed25519 + SHA-256
Dépôt de code / politique	GitLab API v4
Qualité	SonarScanner(s) + SonarQube Web API
Sécurité	JFrog Xray API / outils CLI pris en charge
IA	API Mistral / agents organisationnels
Persistance	PostgreSQL, facultatif au stade prototype
Distribution	VSIX via dépôt interne sécurisé

Pour compiler le cœur Java : JDK 21 et Maven compatible avec la version retenue de Spring Boot. Pour compiler l'extension : Node.js, npm, TypeScript et les outils VS Code Extension appropriés. La présence de Git est nécessaire pour les opérations GitFlow. Les SDK cibles (Python, .NET, Angular/Node) dépendent du starter kit utilisé ; ils ne sont pas tous nécessaires au développement du noyau Java.

Portée initiale recommandée : VS Code Desktop sur Windows et Linux. La version Web de VS Code devra être étudiée séparément car elle ne permet pas d'exécuter librement le même processus Java local qu'une extension Desktop.

10. Lots de réalisation et stratégie de tests

Lot	Objet	Livrables prioritaires
1	Socle et gouvernance	Extension, liaison Java, identité GitLab, politique signée, tests de refus
2	Starter Kit	Initialisation Java, Python, .NET et Angular
3	IA Code Generator	Authentification, contexte, chat, agents, tokens, quotas
4	Quality + Security	Scans Sonar/Xray, rapports, prompts et diff de correction

Lot	Objet	Livrables prioritaires
5	GitFlow + industrialisation	Git/MR, audit, packaging, CI, tests de sécurité

Jeu de tests de sécurité obligatoire

- Projet GitLab inconnu, non autorisé ou ne correspondant pas à l'instance cible.
- Politique absente, expirée, modifiée, mal signée ou utilisant une version antérieure.
- Changement de remote, de branche et de workspace ; workspace multi-racines.
- Tentative d'appel direct de la gateway alors que le chat est grisé.
- Tentative d'envoyer du code d'un projet non autorisé via le contexte d'un projet autorisé.
- Révocation de droits entre deux appels, quota dépassé et perte de connectivité.
- Exfiltration de secrets via prompt injection dans un fichier source ou un rapport d'analyse.
- Permissions d'agent : modification non autorisée de fichiers, commandes et opérations Git.

Les tests doivent couvrir les modules Java et la gateway, avec des tests d'intégration contre des instances de qualification GitLab/Sonar/JFrog. Les secrets réels ne doivent pas être présents dans les environnements de test ni les logs.

11. Décisions d'architecture

1 — Java porte les fonctions métier et la ségrégation

TypeScript se limite à l'intégration native dans VS Code et à la présentation des cinq vues. Les règles organisationnelles, le traitement des rapports, les politiques, le contexte et l'orchestration sont réalisés en Java.

2 — Les comptes techniques restent côté serveur

Les jetons de service et clés IA ne sont pas intégrés au VSIX. Le serveur conserve les secrets dans un coffre et applique la rotation, les droits minimaux et l'audit. Les opérations sur le dépôt (push, MR) respectent l'identité et les droits du développeur.

3 — Le blocage IA est imposé par la gateway

Griser une vue n'est pas une mesure de sécurité suffisante. La gateway applique systématiquement les permissions et la traçabilité, y compris lorsque l'appel IA est initié indirectement par Sonar, Xray ou GitFlow.

RECOMMANDATION FINALE La première version opérationnelle devrait comprendre les cinq vues de l'extension, le moteur Java de ségrégation, la lecture de politique GitLab signée et l'onglet Starter Kit. Les fonctions IA sont ajoutées ensuite au-dessus de ce socle de confiance.

12. Sources et documentation officielle

Ressources techniques de référence à consulter pour le choix final des versions et la mise en œuvre :

VS Code Extension API — https://code.visualstudio.com/api

VS Code Web Extensions — https://code.visualstudio.com/api/extension-guides/web-extensions

GitLab REST API — https://docs.gitlab.com/api/

GitLab Repository Files API — https://docs.gitlab.com/api/repository_files/

GitLab Merge Requests API — https://docs.gitlab.com/api/merge_requests/

GitLab Service Accounts — https://docs.gitlab.com/user/profile/service_accounts/

SonarQube Server documentation — https://docs.sonarsource.com/sonarqube-server/

JFrog Security / Xray documentation — https://docs.jfrog.com/security/

Mistral AI documentation — https://docs.mistral.ai/

Spring Boot Reference — https://docs.spring.io/spring-boot/reference/

Oracle Java Security / Signature — https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/security/Signature.html

Note : les identifiants de projets, URL organisationnelles, quotas et données d’interface présents dans ce document sont des exemples de conception, non des paramètres effectifs. La compatibilité des versions et les droits d’accès devront être confirmés dans l’environnement de l’organisation.